NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

Official Study Material • Java Programming & OOP (JAVA101) • Unit 3: Exception Handling & Multithreading

Course Code:	JAVA101	**Regulation:**	R20 / R23 Autonomous
Subject:	Java Programming & OOP	**Unit:**	Unit 3 - Exception Handling & Multithreading

NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC STUDY MATERIAL: JAVA101 JAVA PROGRAMMING & OOP

UNIT 3: EXCEPTION HANDLING & MULTITHREADING

Regulation: R20 / R23 Autonomous Syllabus | Credits: 4

1. JAVA EXCEPTION HANDLING FRAMEWORK:

Exceptions are abnormal runtime conditions disrupting program flow.

- Throwable Hierarchy: Throwable -> Error (Out of Memory, StackOverflow) and Exception.

* Checked Exceptions (Compile-time): Checked at compile time (IOException, SQLException, ClassNotFoundException).

* Unchecked Exceptions (Runtime): Subclasses of RuntimeException (NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException).

- Exception Keywords:

* try: Encloses code that may throw an exception.

* catch: Handles specific exception type. Multiple catch blocks must order subclasses before superclasses.

* finally: Always executes regardless of whether an exception occurred, handled, or unhandled (cleanup code).

* throw: Explicitly throws an exception object: throw new ArithmeticException("Invalid");

* throws: Declares exceptions in method signature that method may pass to caller.

- Custom Exceptions: User-defined classes extending Exception (checked) or RuntimeException (unchecked).

2. MULTITHREADING & THREAD LIFECYCLE:

A thread is a lightweight sub-process executing concurrently within the shared memory space of a process.

- Thread Lifecycle States:

1. New: Created but start() not called.

2. Runnable: Ready to run in OS thread pool.

3. Running: CPU executing thread's run() method.

4. Blocked/Waiting: Waiting for monitor lock or sleep/wait interval.

5. Terminated (Dead): run() method completes execution.

- Creating Threads:

* Extending Thread class and overriding run().

* Implementing Runnable interface (preferred for supporting class inheritance).

3. SYNCHRONIZATION & INTER-THREAD COMMUNICATION:

- Thread Interference & Race Condition: Occurs when multiple threads modify shared mutable state concurrently.

- Synchronization: synchronized keyword locks object monitor, ensuring mutual exclusion.

- Inter-Thread Communication: Methods of Object class:

* wait(): Releases object lock and waits until notified.

* notify(): Wakes up a single thread waiting on this object.

* notifyAll(): Wakes up all waiting threads.

4. PRACTICAL CODE EXAMPLE:

```
// Java Program demonstrating Custom Exception and Multithreading with Runnable
class InsufficientFundsException extends Exception {
public InsufficientFundsException(String msg) {
super(msg);
}
}

class BankAccount {
private double balance = 5000.0;

public synchronized void withdraw(double amount) throws InsufficientFundsException {
if (amount > balance) {
throw new InsufficientFundsException("Withdrawal of " + amount + " failed! Balance is " + balance);
}
balance -= amount;
System.out.println(Thread.currentThread().getName() + " withdrew " + amount + ". Remaining: " + balance);
}
}

public class ThreadExceptionDemo {
public static void main(String[] args) {
BankAccount account = new BankAccount();

Runnable task = () -> {
try {
account.withdraw(3000.0);
} catch (InsufficientFundsException e) {
System.err.println(Thread.currentThread().getName() + " caught: " + e.getMessage());
}
};

Thread t1 = new Thread(task, "ATM-1");
Thread t2 = new Thread(task, "ATM-2");
t1.start();
t2.start();
}
}
```

Output:

ATM-1 withdrew 3000.0. Remaining: 2000.0

ATM-2 caught: Withdrawal of 3000.0 failed! Balance is 2000.0

5. IMPORTANT EXAM QUESTIONS & ANSWERS:

Q1: Explain the purpose of 'finally' block and describe scenarios where 'finally' does not execute.

Answer: finally executes always to close resources like files or database connections. It does NOT execute only if System.exit(0) is called, JVM crashes, or fatal hardware/OS failure occurs.

Q2: Compare extending Thread class vs implementing Runnable interface.

Answer: Implementing Runnable is superior because Java prohibits multiple inheritance of classes (a class extending Thread cannot extend another class), promotes separation of concerns (task logic vs thread execution), and is compatible with modern Thread Pools (ExecutorService).